

# UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Facultad de Contaduría y Administración C-1  
Ingeniería en Desarrollo y Tecnologías de Software

## Automatización de notificaciones por correo con n8n

Adaptador N8nWebhookAdapter sobre el puerto EmailServicePort y flujo de n8n para el  
Sistema de Comercio Electrónico 5M

REPOSITORIO DEL CÓDIGO FUENTE

[github.com/LuisAGA-1/ecommerce-5m](https://github.com/LuisAGA-1/ecommerce-5m)

|                     |                                                  |
|---------------------|--------------------------------------------------|
| Alumno:             | Luis Angel Alegria Gomez                         |
| Semestre y grupo:   | 5.º semestre · Grupo M                           |
| Entorno de pruebas: | Local (localhost) · WSL2 Ubuntu 24.04 · n8n 5678 |
| Fecha:              | 9 de octubre de 2026                             |
| Lugar:              | Tuxtla Gutiérrez, Chiapas                        |

## 1. Objetivo y alcance

Este reporte documenta la integración de n8n, una herramienta de automatización de flujos de trabajo, con el subsistema de notificaciones por correo del e-commerce. Cuando un cliente genera un pedido, el sistema envía el comprobante con las instrucciones de pago al cliente y un aviso de nuevo pedido al administrador. Ahora ese envío puede delegarse a un flujo de n8n sin modificar el núcleo de la aplicación.

El trabajo se apoya en la actividad anterior, en la que se creó el puerto de salida EmailServicePort y el adaptador NodemailerAdapter. En esta entrega se añade:

- **N8nWebhookAdapter** segundo adaptador del mismo puerto, que publica el evento pedido.creado en un webhook de n8n.
- **EmailServiceConRespaldo** adaptador compuesto que intenta primero con n8n y, si falla, envía con Nodemailer.
- **crearServicioNotificaciones** fábrica que elige el adaptador según la variable EMAIL\_PROVIDER.
- **Flujo de n8n** workflow importable (n8n/01-pedido-creado-notificaciones.json) que valida el secreto y envía los dos correos.

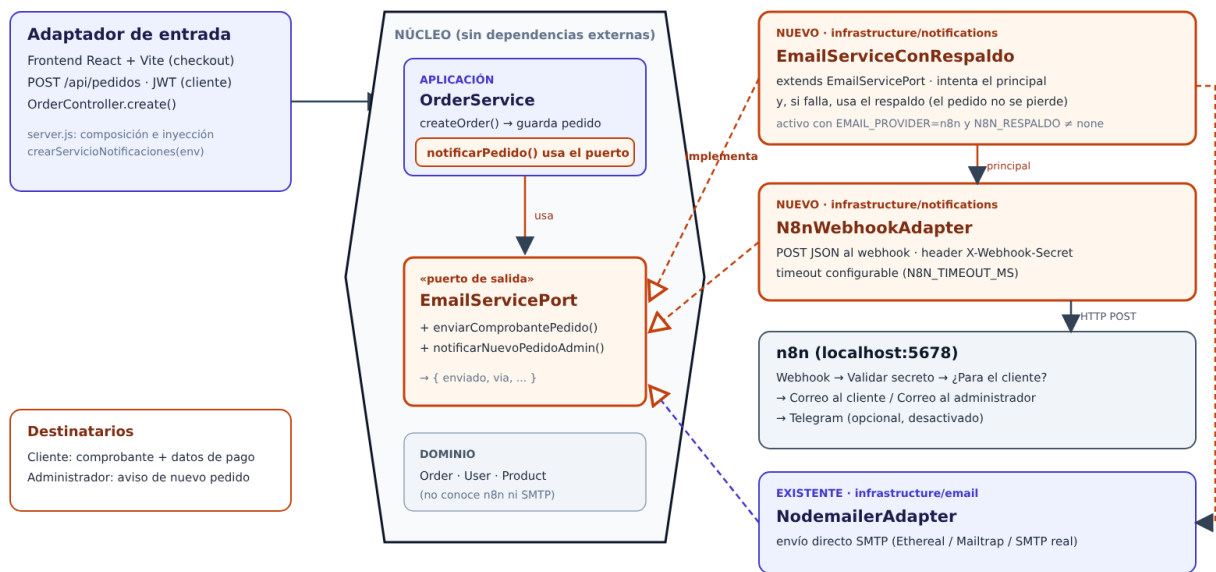
*Alcance: este reporte cubre únicamente el envío de correos y la automatización con n8n. El dashboard analítico y sus endpoints de reportes se entregarán por separado y no se describen aquí.*

## 2. Arquitectura

La arquitectura hexagonal permite el cambio porque OrderService depende solo del puerto EmailServicePort. Nodemailer, n8n o el respaldo son detalles de infraestructura que se conectan en un único lugar: la composición de server.js, mediante la fábrica crearServicioNotificaciones.

### Arquitectura hexagonal - notificaciones con n8n

Un mismo puerto (EmailServicePort), tres adaptadores intercambiables



Naranja: nuevo en esta actividad · Azul: existente · Línea punteada: implementa el puerto

Figura 1. Arquitectura con el puerto EmailServicePort y sus adaptadores (en naranja lo nuevo en esta actividad).

## 2.1 Selección del adaptador

| EMAIL_PROVIDER             | Adaptador conectado          | Comportamiento                                                              |
|----------------------------|------------------------------|-----------------------------------------------------------------------------|
| ethereal   mailtrap   smtp | NodemailerAdapter            | Envío directo por SMTP, como en la actividad anterior.                      |
| n8n                        | N8nWebhookAdapter + respaldo | Publica en n8n. Si falla, usa el proveedor de N8N_RESPALDO.                 |
| n8n con N8N_RESPALDO=none  | N8nWebhookAdapter solo       | Sin respaldo: si n8n falla, el pedido se guarda pero el correo no se envía. |
| none                       | Ninguno                      | El pedido se crea sin notificaciones.                                       |

## 3. Contrato del evento hacia n8n

El backend envía una petición HTTP al webhook de n8n por cada destinatario (dos por pedido: cliente y administrador).

| Elemento           | Valor                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------|
| Método y URL       | POST N8N_WEBHOOK_URL (por defecto <code>http://localhost:5678/webhook/pedido-creado</code> ) |
| Content-Type       | application/json                                                                             |
| X-Webhook-Secret   | Valor de N8N_WEBHOOK_SECRET. Si no está definido, el header no se envía.                     |
| Tiempo de espera   | N8N_TIMEOUT_MS (5000 ms por defecto); al vencer se cancela la petición con AbortController.  |
| Respuesta esperada | Cualquier código 2xx. Otro código se trata como fallo y activa el respaldo.                  |

### 3.1 Cuerpo de la petición

```
{
  "evento": "pedido.creado",
  "destinatario": "cliente",           // o "admin"
  "para": "cliente@gmail.com",
  "tienda": "Tienda 5M",
  "pedido": {
    "id": 17, "folio": "PED-000017", "fecha": "2026-10-08T16:28:00.000Z",
    "estado": "PENDIENTE", "estadoEtiqueta": "Pendiente de pago", "total": 21099,
    "detalles": [{ "producto": "Laptop Lenovo IdeaPad", "cantidad": 1,
      "precioUnitario": 16500, "subtotal": 16500 }]
  },
  "cliente": { "id": 12, "nombre": "cliente", "email": "cliente@gmail.com" },
  "pago": { "banco": "BBVA México", "clabe": "012100001234567895", "referencia": "PED-000017" },
  "correo": { "subject": "Comprobante de pedido PED-000017 – Pendiente de pago",
    "html": "<!doctype html>...", "text": "..." },
  "enviadoEn": "2026-10-08T16:28:01.120Z"
}
```

Para el administrador, el cuerpo cambia en "destinatario": "admin", "para" (ADMIN\_EMAIL) y se agrega "urlPanel"; no incluye el bloque "pago". El correo ya viene armado en "correo": n8n no construye el HTML, solo decide por dónde enviarlo, de modo que las plantillas siguen en un solo lugar del backend.

## 3.2 Variables de entorno (backend/.env)

| Variable           | Valor de ejemplo                                | Descripción                                       |
|--------------------|-------------------------------------------------|---------------------------------------------------|
| EMAIL_PROVIDER     | n8n                                             | Proveedor de notificaciones.                      |
| N8N_WEBHOOK_URL    | http://localhost:5678/webhook/<br>pedido-creado | URL de producción del webhook.                    |
| N8N_WEBHOOK_SECRET | cambia-este-secreto                             | Secreto compartido con el nodo «Validar secreto». |
| N8N_TIMEOUT_MS     | 5000                                            | Tiempo máximo de espera.                          |
| N8N_RESPALDO       | ethereal   mailtrap   smtp   none               | Adaptador de respaldo si n8n falla.               |

## 4. Código fuente

Todo el código está en <https://github.com/LuisAGA-1/ecommerce-5m>. Archivos de esta actividad:

| Archivo                                                                         | Capa            | Tipo         | Descripción                                                                                     |
|---------------------------------------------------------------------------------|-----------------|--------------|-------------------------------------------------------------------------------------------------|
| backend/src/infrastructure/<br>notifications/<br>N8nWebhookAdapter.js           | Infraestructura | Nuevo        | Implementa el puerto publicando pedido.creado en el webhook (secreto, timeout, errores claros). |
| backend/src/infrastructure/<br>notifications/<br>EmailServiceConRespaldo.js     | Infraestructura | Nuevo        | Adaptador compuesto: principal (n8n) y respaldo (Nodemailer).                                   |
| backend/src/infrastructure/<br>notifications/<br>crearServicioNotificaciones.js | Infraestructura | Nuevo        | Fábrica que elige el adaptador según EMAIL_PROVIDER.                                            |
| backend/src/interfaces/<br>server.js                                            | Interfaces      | Modificado   | Construye el servicio con la fábrica e inyecta el resultado en OrderService.                    |
| n8n/01-pedido-creado-<br>notificaciones.json                                    | Automatización  | Nuevo        | Flujo de n8n importable.                                                                        |
| n8n/docker-compose.yml                                                          | Automatización  | Nuevo        | Levanta n8n con Docker (puerto 5678, zona horaria de México).                                   |
| backend/tests/<br>n8n.test.js                                                   | Pruebas         | Nuevo        | Pruebas del adaptador, del respaldo y de la fábrica.                                            |
| docs/N8N.md<br>· .env.example                                                   | Documentación   | Nuevo / Mod. | Guía de puesta en marcha y variables de entorno.                                                |

**OrderService y EmailServicePort no cambiaron: el núcleo sigue llamando a enviarComprobantePedido() y notificarNuevoPedidoAdmin().**

### 4.1 N8nWebhookAdapter (publicación del evento)

```
class N8nWebhookAdapter extends EmailServicePort {
  constructor(config, fetchFn = globalThis.fetch) {
    super();
    if (!config || !config.webhookUrl)
      throw new Error("N8N_WEBHOOK_URL es obligatorio para usar el adaptador de n8n");
    this.config = config; this.fetch = fetchFn; // fetch inyectable para pruebas
  }

  async publicar(cuerpo) {
```

```

const controlador = new AbortController();
const temporizador = setTimeout(() => controlador.abort(),
    Number(this.config.timeoutMs || 5000));
try {
    const respuesta = await this.fetch(this.config.webhookUrl, {
        method: "POST",
        headers: { "Content-Type": "application/json",
            ...(this.config.secreto ? { "X-Webhook-Secret": this.config.secreto } : {}) },
        body: JSON.stringify(cuerpo), signal: controlador.signal });
    if (!respuesta.ok) throw new Error(`n8n respondió ${respuesta.status}`);
    return { enviado: true, via: "n8n", estadoHttp: respuesta.status };
} catch (error) {
    if (error.name === "AbortError") throw new Error("n8n no respondió a tiempo");
    if (error.cause?.code) throw new Error(`No se pudo conectar con n8n ($
{error.cause.code})`);
    throw error;
} finally { clearTimeout(temporizador); }
}
// enviarComprobantePedido() y notificarNuevoPedidoAdmin() arman el correo con las
// plantillas existentes y llaman a publicar() con destinatario "cliente" o "admin".
}

```

## 4.2 EmailServiceConRespaldo (tolerancia a fallos)

```

async conRespaldo(metodo, datos) {
    try {
        return await this.principal[metodo](datos);
    } catch (error) {
        if (!this.respaldo) throw error;
        this.registrar.warn(`[notificaciones] Falló el adaptador principal (${error.message});
usando respaldo.`);
        const resultado = await this.respaldo[metodo](datos);
        return { ...resultado, respaldo: true, motivoRespaldo: error.message };
    }
}

```

## 4.3 Fábrica y composición

```

function crearServicioNotificaciones(env = process.env) {
    const proveedor = (env.EMAIL_PROVIDER || "ethereal").toLowerCase();
    if (proveedor === "none") return null;
    const base = NodemailerAdapter.configDesdeEntorno(env);
    if (proveedor !== "n8n") return new NodemailerAdapter(base);

    const n8n = new N8nWebhookAdapter({ ...base, webhookUrl: env.N8N_WEBHOOK_URL,
        secreto: env.N8N_WEBHOOK_SECRET, timeoutMs: Number(env.N8N_TIMEOUT_MS || 5000) });
    const respaldo = (env.N8N_RESPALDO || "ethereal").toLowerCase();
    if (respaldo === "none") return n8n;
    return new EmailServiceConRespaldo(n8n, new NodemailerAdapter({ ...base, proveedor:
    respaldo }));
}

// server.js (única línea que decide la implementación)
const emailService = crearServicioNotificaciones(process.env);

```

Un fallo de notificación nunca revierte el pedido: `OrderService.notificarPedido()` usa `Promise.allSettled`, registra el error y devuelve el resultado de cada envío al frontend.

## 5. Flujo de trabajo en n8n

El archivo n8n/01-pedido-creado-notificaciones.json contiene el flujo «E-commerce 5M – Pedido creado (notificaciones)».

| Nodo                         | Tipo              | Función                                                        |
|------------------------------|-------------------|----------------------------------------------------------------|
| Webhook pedido creado        | Webhook (POST)    | Recibe el evento en /webhook/pedido-creado.                    |
| Validar secreto              | IF                | Compara el header X-Webhook-Secret con el secreto del backend. |
| Secreto inválido (ignorar)   | No Operation      | Descarta las peticiones con secreto incorrecto.                |
| ¿Para el cliente?            | IF                | Separa el evento según destinatario (cliente o admin).         |
| Correo al cliente            | Send Email (SMTP) | Envía correo.subject y correo.html al campo para.              |
| Correo al administrador      | Send Email (SMTP) | Envía el aviso de nuevo pedido al administrador.               |
| Telegram al admin (opcional) | Telegram          | Aviso adicional; viene desactivado por defecto.                |

### 5.1 Puesta en marcha

1. **Levantar n8n** con «npx n8n» (Node 20 o superior) o con «docker compose up -d» dentro de la carpeta n8n. Se abre en <http://localhost:5678>.
2. **Crear la credencial SMTP** en Credentials → Add credential → SMTP, con la misma cuenta de Ethereum del backend (host smtp.ethereal.email, puerto 587, SSL/TLS desactivado).
3. **Importar el flujo** Workflows → Import from File → n8n/01-pedido-creado-notificaciones.json. Asignar la credencial SMTP a los dos nodos de correo y verificar que el secreto de «Validar secreto» coincida con N8N\_WEBHOOK\_SECRET.
4. **Activar el flujo** con el switch Active. Solo así escucha en la URL de producción /webhook/ (la URL /webhook-test/ funciona únicamente mientras se presiona Listen for test event).
5. **Configurar el backend** EMAIL\_PROVIDER=n8n y las variables N8N\_\* en backend/.env; reiniciar con «npm run dev».

## 6. Evidencias de funcionamiento

### 6.1 Ejecución exitosa del flujo en n8n

Tras crear un pedido, n8n registra la ejecución con todos los nodos en verde: el webhook recibió el evento, el secreto fue válido y se enviaron los correos al cliente y al administrador. El nodo de Telegram aparece desactivado, como se esperaba.

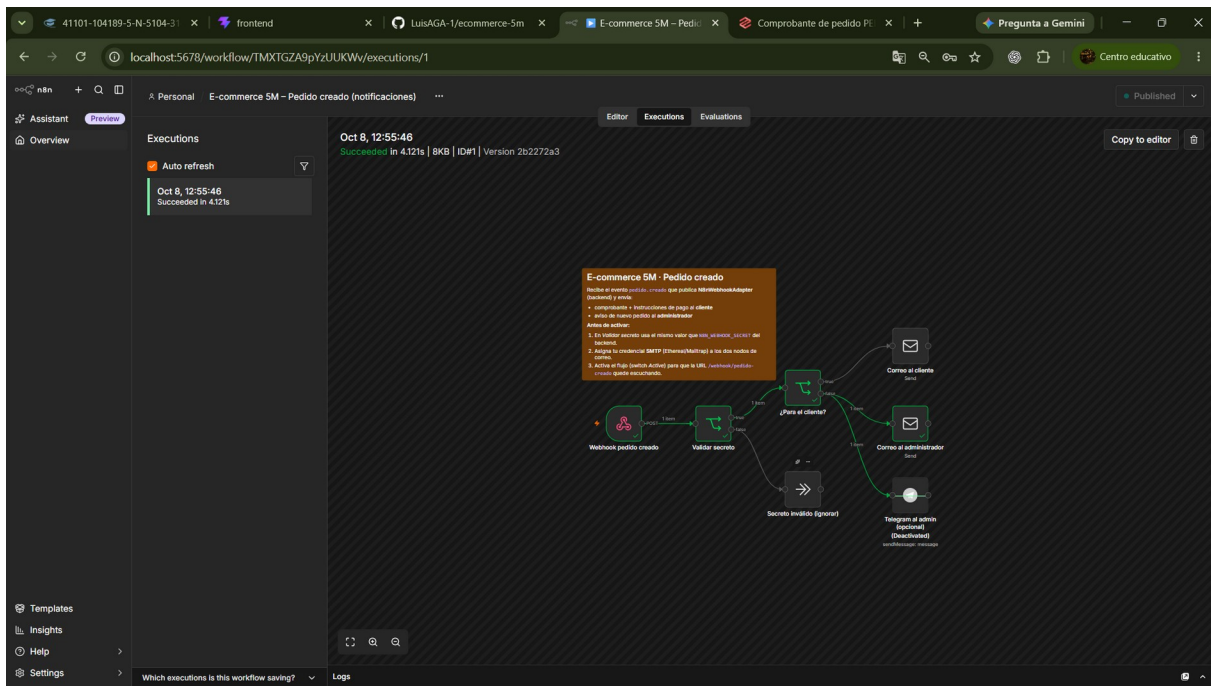


Figura 2. Ejecución #1 en n8n (8 de octubre, 12:55:46): estado Succeeded en 4.121 s.

## 6.2 Correo recibido por el cliente

El pedido PED-000222 se creó desde la versión desplegada en AWS (<http://50.16.147.230>). La pantalla de confirmación muestra el folio y el estado «Pendiente de pago», y el comprobante llegó a la bandeja de Ethereum con el desglose, el total y las instrucciones de transferencia.

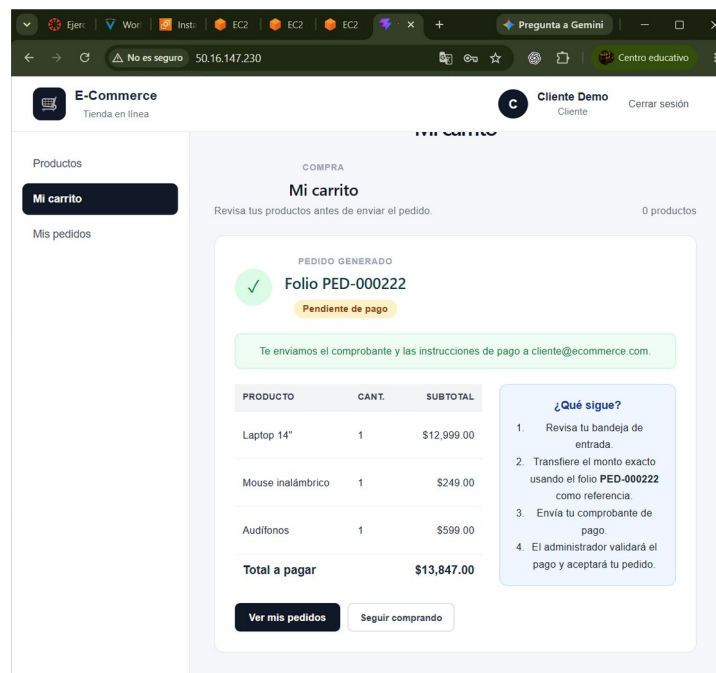


Figura 3. Confirmación del pedido PED-000222 en el frontend: folio, estado y siguientes pasos.

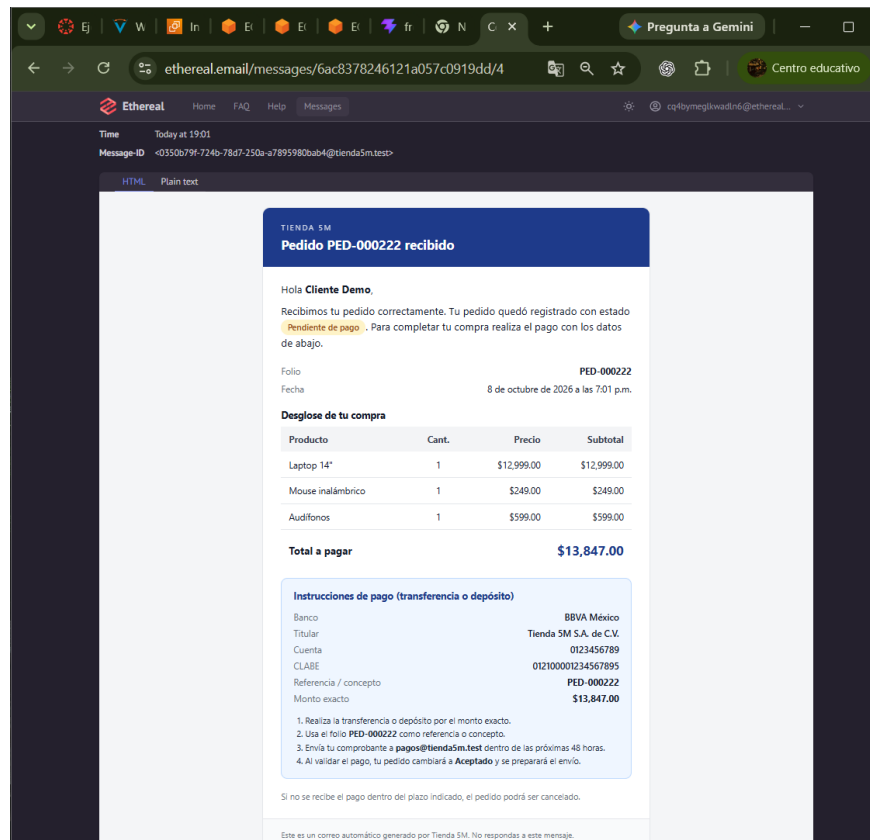


Figura 4. Comprobante «Pedido PED-000222 recibido» en la bandeja de Ethereum; productos, total de \$13,847.00 y datos bancarios.

### 6.3 Comportamiento cuando n8n no está disponible

Con la configuración de la figura 5 (EMAIL\_PROVIDER=n8n y N8N\_RESPALDO=none) se apagó n8n y se creó un pedido. El backend no pudo conectar con el webhook, registró el error ECONNREFUSED con un mensaje claro y el pedido se guardó de todos modos. El frontend avisó al cliente que el correo no pudo enviarse (figura 6).

```
luis_angel@AMA-Mod-1: ~/w x + v - □ x
N8N_WEBHOOK_URL=http://localhost:5678/webhook/pedido-creado
N8N_WEBHOOK_SECRET=cambia-este-secreto
N8N_TIMEOUT_MS=5000
N8N_RESPALDO=none
EOF

# 3. Verificar
grep -n "EMAIL_PROVIDER\|N8N_" backend/.env
node -e "require('dotenv').config({path: 'backend/.env', quiet: true}); console.log(process.env.EMAIL_PROVIDER, process.env.N8N_RESPALDO)"
10:EMAIL_PROVIDER=n8n
25:N8N_WEBHOOK_URL=http://localhost:5678/webhook/pedido-creado
26:N8N_WEBHOOK_SECRET=cambia-este-secreto
27:N8N_TIMEOUT_MS=5000
28:N8N_RESPALDO=none
n8n none
luis_angel@AMA-Mod-1:~/webapp$ npm run dev

> webapp@1.0.0 dev
> node --watch backend/src/interfaces/server.js

◇ injected env (25) from backend/.env // tip: % enable debugging { debug: true }
Servidor escuchando en el puerto 3000
[pedido] No se envió el comprobante al cliente: Error: No se pudo conectar con n8n (ECONNREFUSED). ¿Está corriendo en http://localhost:5678/webhook/pedido-creado?
    at N8nWebhookAdapter.publicar (/home/luis_angel/webapp/backend/src/infrastructure/notifications/N8nWebhookAdapter.js:99:15)
    at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
    at async Promise.allSettled (index 0)
    at async OrderService.notificarPedido (/home/luis_angel/webapp/backend/src/application/orders/OrderService.js:41:30)
    at async OrderService.createOrder (/home/luis_angel/webapp/backend/src/application/orders/OrderService.js:145:26)
    at async OrderController.create (/home/luis_angel/webapp/backend/src/interfaces/controllers/orderController.js:16:8)
[pedido] No se notificó al administrador: Error: No se pudo conectar con n8n (ECONNREFUSED). ¿Está corriendo en http://localhost:5678/webhook/pedido-creado?
    at N8nWebhookAdapter.publicar (/home/luis_angel/webapp/backend/src/infrastructure/notifications/N8nWebhookAdapter.js:99:15)
    at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
    at async Promise.allSettled (index 1)
    at async OrderService.notificarPedido (/home/luis_angel/webapp/backend/src/application/orders/OrderService.js:41:30)
    at async OrderService.createOrder (/home/luis_angel/webapp/backend/src/application/orders/OrderService.js:145:26)
    at async OrderController.create (/home/luis_angel/webapp/backend/src/interfaces/controllers/orderController.js:16:8)
```

Figura 5. Terminal del backend: configuración N8N\_\* y errores ECONNREFUSED al notificar cliente y administrador.

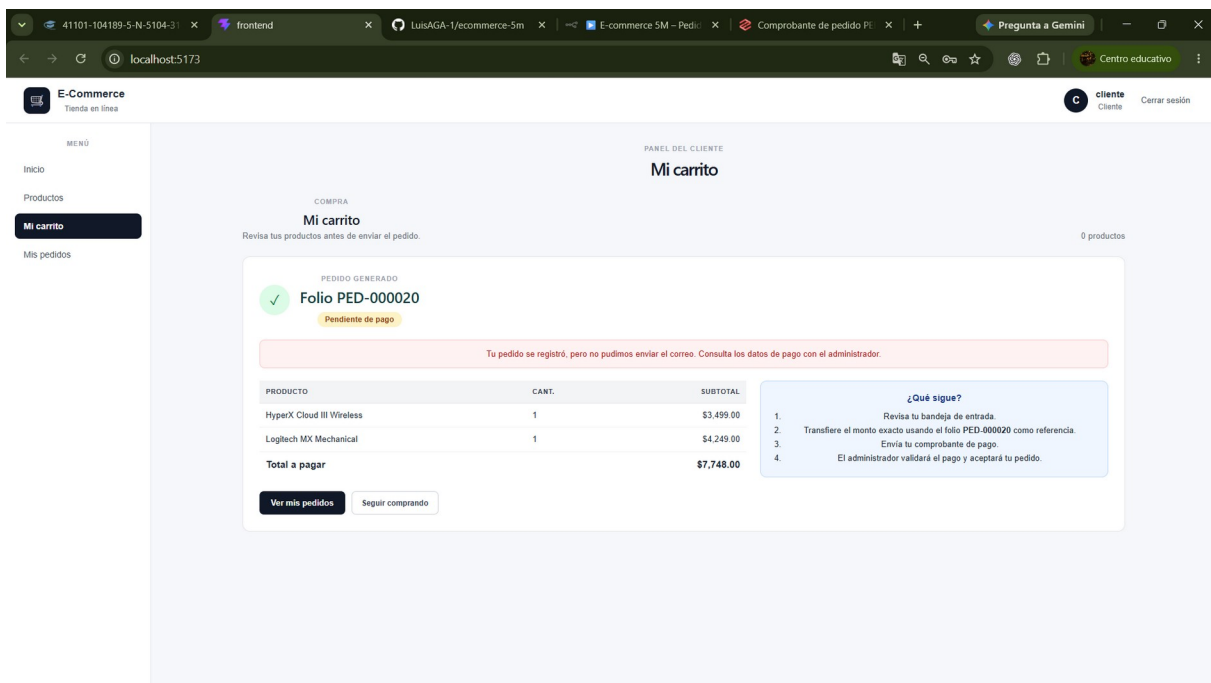


Figura 6. Pedido PED-000020 registrado con el aviso «no pudimos enviar el correo. Consulta los datos de pago con el administrador».

Con N8N\_RESPALDO=ethereal (valor recomendado), el mismo escenario registra «Falló el adaptador principal... usando respaldo» y el correo llega igual por Nodemailer.

## 7. Pruebas automatizadas

Las pruebas se ejecutan con «npm test» (node:test). No necesitan n8n, SMTP ni base de datos: usan un fetch simulado y adaptadores falsos en memoria. Las pruebas de esta actividad son:

| Prueba                             | Verifica                                                                              |
|------------------------------------|---------------------------------------------------------------------------------------|
| Implementa el puerto               | N8nWebhookAdapter es una instancia de EmailServicePort.                               |
| Exige la URL del webhook           | Sin webhookUrl el constructor lanza error.                                            |
| Publica el comprobante del cliente | Envía datos del pedido, correo armado y el header X-Webhook-Secret.                   |
| Publica el aviso al administrador  | El evento llega con destinatario «admin».                                             |
| Sin secreto                        | No se envía el header X-Webhook-Secret.                                               |
| Respuesta HTTP de error            | Un código no 2xx se reporta como fallo.                                               |
| Timeout                            | Si n8n no responde, la petición se cancela.                                           |
| Respaldo no usado                  | Con n8n disponible no se invoca el adaptador de respaldo.                             |
| Respaldo usado                     | Si n8n falla, se envía por el respaldo e indica el motivo.                            |
| OrderService con n8n               | El pedido se crea y se notifica sin cambios en el núcleo.                             |
| Fábrica                            | EMAIL_PROVIDER elige el adaptador correcto (Nodemailer, n8n, n8n con respaldo, none). |

Junto con las 6 pruebas de la actividad anterior (notificaciones.test.js: envío al crear pedido, pedido conservado si el correo falla, sin adaptador, puerto sin implementar y las dos plantillas), cubren el subsistema de correo completo.

## 8. Seguridad y manejo de fallos

- **Secreto del webhook** n8n descarta las peticiones cuyo X-Webhook-Secret no coincide con el configurado.
- **Sin credenciales en el código** la URL, el secreto y las cuentas SMTP viven en backend/.env (fuera de GitHub); las credenciales SMTP de n8n se guardan en n8n y no se exportan con el flujo.
- **Tiempo de espera** la petición a n8n se cancela a los N8N\_TIMEOUT\_MS, para que el checkout no se quede esperando.
- **Pedido primero, correo después** un fallo de notificación no revierte el pedido; se registra y se informa al usuario.
- **Respaldo** con N8N\_RESPALDO configurado, la caída de n8n no interrumpe el envío de correos.

## 9. Conclusión

La integración con n8n se logró sin tocar el núcleo de la aplicación: bastó implementar un segundo adaptador del puerto EmailServicePort y cambiar la composición en server.js. Esto demuestra el valor de la arquitectura hexagonal, porque el sistema puede enviar correos directamente con Nodemailer, delegarlos a n8n o combinar ambos con respaldo solo con cambiar una variable de entorno. Además, n8n abre la puerta

a nuevas automatizaciones, como avisos por Telegram, sin modificar el backend. Las evidencias muestran el flujo ejecutándose correctamente, el comprobante recibido por el cliente y el manejo ordenado del fallo cuando n8n no está disponible.